

UNIVERSIDAD DE MURCIA
FACULTAD DE INFORMÁTICA

Redes de Comunicaciones
Proyecto NanoFiles



Juan Martínez Moreno
Grupo: 3, Subgrupo: 3
juan.martinezm@um.es
23828088L

Julio Maylin Martínez
Grupo: 3, Subgrupo: 3
julio.maylinm@um.es
49246995Q

Mayo de 2026

Índice

1. Introducción	3
2. Protocolo con el directorio	3
2.1. Descripción general	3
2.2. Operaciones definidas	3
2.3. Campos utilizados	4
2.4. Ejemplos de mensajes	4
2.4.1. Comando <code>ping</code>	4
2.4.2. Comando <code>serve</code>	4
2.4.3. Comando <code>dirfiles</code>	4
2.4.4. Comando <code>peers</code>	5
2.4.5. Comando <code>dirdl</code>	5
2.4.6. Comando <code>quit</code>	5
2.5. Autómata del peer como cliente del directorio	5
2.6. Autómata del servidor de directorio	6
3. Protocolo peer-to-peer sobre TCP	7
3.1. Descripción general	7
3.2. OpCodes definidos	7
3.3. Comando <code>peerfiles</code>	7
3.3.1. Solicitud <code>FILE_LIST</code>	7
3.3.2. Respuesta <code>FILE_LIST</code>	8
3.4. Comando <code>peerdl</code>	8
3.4.1. Solicitud <code>GET_FILE_HASH</code>	8
3.4.2. Respuesta <code>SEND_FILE</code> con metadatos	8
3.4.3. Solicitud <code>GET_CHUNK</code>	9
3.4.4. Respuesta <code>SEND_FILE</code> con datos	9
3.5. Autómata del peer como cliente TCP	9
3.6. Autómata del peer como servidor TCP	11
4. Funcionalidad implementada	12
4.1. Funcionalidad obligatoria	12
4.2. Mejoras implementadas	13
5. Capturas de ejecución y Wireshark	13
6. Pruebas realizadas	14
7. Conclusiones	14

1. Introducción

El objetivo de esta práctica es diseñar e implementar un sistema de compartición y transferencia de ficheros denominado **nanoFiles**. El sistema se compone de un servidor de directorio, ejecutado mediante el programa **Directory**, y de varios pares o *peers*, ejecutados mediante el programa **NanoFiles**.

El servidor de directorio mantiene un registro de pares registrados, asociando el *nickname* de cada par con su puerto. Además, el directorio tiene una carpeta compartida propia. Por su parte, cada *peer* puede actuar simultáneamente como cliente, consultando el directorio o descargando ficheros, y como servidor de ficheros, atendiendo peticiones TCP de otros pares.

La comunicación del proyecto se divide en dos protocolos de aplicación:

- Un protocolo UDP para la comunicación *peer*-directorio.
- Un protocolo TCP para la comunicación directa entre *peers*.

En la implementación se ha usado el identificador de protocolo **123456789A**. Este identificador permite comprobar que el directorio y los *peers* pertenecen al mismo grupo y usan un protocolo compatible.

2. Protocolo con el directorio

2.1. Descripción general

La comunicación con el directorio se realiza mediante UDP. Como UDP no garantiza la entrega de los datagramas, el cliente implementa un mecanismo de parada y espera. Cada solicitud se envía al directorio y el cliente espera una respuesta durante un tiempo máximo. Si no llega respuesta, se retransmite la misma solicitud hasta un máximo de cinco intentos.

El directorio escucha en el puerto UDP **6868**. Los mensajes son textuales y siguen el formato:

```
campo:valor
campo:valor
...
```

El primer campo de cada mensaje es siempre **operation**, que indica el tipo de operación solicitada o respondida.

2.2. Operaciones definidas

Operación	Descripción
ping	Comprueba que el directorio está activo y usa un protocolo compatible.
welcome	Respuesta positiva del directorio.
serve	Registra al <i>peer</i> como servidor de ficheros.
dirfiles	Solicita la lista de ficheros disponibles en el directorio.
dirfilesok	Respuesta con la lista de ficheros del directorio.
peers	Solicita el censo de <i>peers</i> registrados.
peersok	Respuesta con el censo de <i>peers</i> .
dirdl	Solicita la descarga de un fichero del directorio mediante una subcadena de hash.
downloadok	Respuesta con datos de un fichero descargado desde el directorio.
quit	Solicita la baja del <i>peer</i> en el directorio.
quitok	Confirma que el <i>peer</i> ha sido eliminado del censo.
error	Indica una operación no válida, un hash ambiguo o un hash inexistente.

2.3. Campos utilizados

Campo	Uso
operation	Tipo de operación del mensaje.
protocol	Identificador de protocolo. Se usa en <code>ping</code> , <code>dirfiles</code> y <code>dirdl</code> .
nickname	Nombre del <i>peer</i> . Se usa en <code>serve</code> y <code>quit</code> .
port	Puerto TCP en el que escucha el servidor de ficheros del <i>peer</i> .
files	Lista de ficheros con formato <code>nombre:tamaño:hash;</code> .
peerlist	Entrada del censo con formato <code>nickname@ip:puerto</code> .
hash	Subcadena de hash solicitada o hash completo del fichero.
filename	Nombre del fichero descargado desde el directorio.
filesize	Tamaño total del fichero.
data	Datos del fichero codificados en Base64.
status	Campo auxiliar para indicar estados de error.

2.4. Ejemplos de mensajes

2.4.1. Comando ping

Solicitud del *peer*:

```
operation:ping
protocol:123456789A
```

Respuesta del directorio si el protocolo es compatible:

```
operation:welcome
```

2.4.2. Comando serve

Solicitud del *peer* para registrarse como servidor:

```
operation:serve
nickname:peer123
port:49152
```

Respuesta del directorio:

```
operation:welcome
```

El directorio almacena internamente la asociación entre el *nickname* recibido y la dirección IP:puertoTCP del par.

2.4.3. Comando dirfiles

Solicitud del *peer*:

```
operation:dirfiles
protocol:123456789A
```

Respuesta del directorio:

```
operation:dirfilesok
files:fichero1.txt:27:hash1;fichero2.txt:40:hash2;
```

Cada entrada contiene nombre, tamaño y hash del fichero.

2.4.4. Comando peers

Solicitud del *peer*:

```
operation:peers
```

Respuesta del directorio:

```
operation:peersok
peerlist:peerA@192.168.1.10:49152
peerlist:peerB@192.168.1.11:49153
```

2.4.5. Comando dirdl

Solicitud del *peer*:

```
operation:dirdl
protocol:123456789A
hash:abc123
```

Respuesta del directorio si el hash identifica un único fichero:

```
operation:downloadok
hash:hash_completo
filename:fichero.txt
filesize:40
data:datos_codificados_en_base64
```

Si la subcadena de hash no identifica ningún fichero, o identifica más de uno, el directorio responde:

```
operation:error
```

2.4.6. Comando quit

Solicitud del *peer*:

```
operation:quit
nickname:peer123
```

Respuesta del directorio:

```
operation:quitok
```

2.5. Autómata del peer como cliente del directorio

El peer, actuando como cliente del directorio, sigue un protocolo de parada y espera sobre UDP. En cada operación se envía una única petición y se espera la respuesta correspondiente. Si no llega respuesta antes de que expire el temporizador, se retransmite la misma petición. Tras agotar el número máximo de intentos, la operación se considera fallida y el peer vuelve al estado de reposo, salvo en el caso de **quit**, en el que la aplicación finaliza.

Los estados del autómata son los siguientes:

- **q0**: estado inicial y de reposo. No hay ninguna petición pendiente con el directorio.
- **qPing**: espera de la respuesta **welcome** tras enviar **ping**.
- **qDirFiles**: espera del primer mensaje **dirfilesok** tras enviar **dirfiles**.

- **qDirFiles+**: recepción de fragmentos adicionales de **dirfilesok** cuando el listado no cabe en un único datagrama.
- **qPeers**: espera de la respuesta **peersok** tras enviar **peers**.
- **qServe**: espera de la respuesta **welcome** tras enviar **serve** para registrar el peer como servidor.
- **qDirDl_i**: espera del bloque **i** de una descarga desde el directorio mediante **dirdl**.
- **qQuit**: espera de la respuesta **quitok** tras enviar **quit**.
- **qFin**: estado final de terminación del programa.

2.6. Autómata del servidor de directorio

El servidor de directorio actúa como servidor UDP en el puerto 6868. Su comportamiento es reactivo: permanece en espera de datagramas, procesa la petición recibida y, en caso de no descartarse el mensaje por la probabilidad de pérdida configurada, envía una respuesta al peer. Después de atender cada petición, vuelve de nuevo al estado de espera.

A diferencia del cliente, el servidor de directorio no mantiene una sesión con cada peer ni conserva el estado de una operación anterior. Por tanto, se modela como un autómata *stateless*: cada datagrama recibido se procesa de forma independiente.

Los estados del autómata son los siguientes:

- **q0**: estado inicial y de espera. El directorio está bloqueado esperando la llegada de un datagrama UDP.
- **qPing**: estado de procesamiento de una petición **ping**. El servidor comprueba si el protocolo indicado por el peer es compatible.
- **qDirFiles**: estado de procesamiento de una petición **dirfiles**. El servidor obtiene la lista de ficheros disponibles en la carpeta compartida del directorio.
- **qDirDl**: estado de procesamiento de una petición **dirdl**. El servidor busca el fichero cuyo hash coincide con la subcadena indicada por el cliente.
- **qServe**: estado de procesamiento de una petición **serve**. El servidor registra el *nickname* del peer junto con su IP y su puerto TCP de escucha.
- **qPeers**: estado de procesamiento de una petición **peers**. El servidor consulta el censo de peers registrados.
- **qQuit**: estado de procesamiento de una petición **quit**. El servidor elimina del censo el peer indicado por su *nickname*.
- **qLoss**: estado auxiliar que representa la pérdida simulada de un datagrama. Si se decide descartar el mensaje según la probabilidad configurada con **-loss**, no se envía respuesta y el servidor vuelve a **q0**.

Las transiciones principales del autómata son:

- Desde **q0**, si se recibe **ping**, se produce la transición **rcv(ping)** hacia **qPing**. Si el protocolo es compatible, el servidor responde con **snd(welcome)** y vuelve a **q0**. Si no lo es, responde con **snd(error)** y vuelve a **q0**.

- Desde **q0**, si se recibe **dirfiles**, se produce la transición **rcv(dirfiles)** hacia **qDirFiles**. El servidor construye el listado de ficheros y responde con **snd(dirfilesok)**. Después vuelve a **q0**.
- Desde **q0**, si se recibe **dirdl**, se produce la transición **rcv(dirdl)** hacia **qDirDl**. Si la subcadena del hash identifica un único fichero, el servidor responde con **snd(downloadok)**. Si no hay coincidencias o la subcadena es ambigua, responde con **snd(error)**. En ambos casos vuelve a **q0**.
- Desde **q0**, si se recibe **serve**, se produce la transición **rcv(serve)** hacia **qServe**. El servidor registra el peer en el censo y responde con **snd(welcome)**. Después vuelve a **q0**.
- Desde **q0**, si se recibe **peers**, se produce la transición **rcv(peers)** hacia **qPeers**. El servidor consulta el censo de peers registrados y responde con **snd(peersok)**. Después vuelve a **q0**.
- Desde **q0**, si se recibe **quit**, se produce la transición **rcv(quit)** hacia **qQuit**. El servidor elimina el peer del censo y responde con **snd(quitok)**. Después vuelve a **q0**.
- Desde cualquiera de los estados de procesamiento, si el datagrama se considera perdido por la probabilidad configurada mediante **-loss**, se pasa a **qLoss**. En ese caso no se envía respuesta y el servidor vuelve directamente a **q0**.

3. Protocolo peer-to-peer sobre TCP

3.1. Descripción general

La comunicación directa entre pares se realiza mediante TCP. Como TCP ya proporciona entrega fiable, ordenada y sin duplicados, el protocolo de aplicación se centra en definir mensajes binarios para listar ficheros y descargar fragmentos.

Todos los mensajes comienzan con un campo **opcode** de un byte. A continuación se escriben los campos específicos de cada operación.

3.2. OpCodes definidos

Valor	Nombre	Descripción
0	INVALID_OPCODE	Código inválido.
1	FILE_NOT_FOUND	El fichero no se ha encontrado o el hash no es único.
2	GET_CHUNK	Solicitud de un fragmento de fichero.
3	FILE_LIST	Solicitud o respuesta con la lista de ficheros.
4	SEND_FILE	Envío de metadatos o de datos de un fichero.
5	GET_FILE_HASH	Consulta de un fichero a partir de una subcadena de hash.

3.3. Comando peerfiles

El comando **peerfiles** <peer_nickname> consulta primero el registro del directorio para obtener la dirección TCP del par. Después abre un socket TCP con ese par y envía una solicitud **FILE_LIST**.

3.3.1. Solicitud FILE_LIST

Campo	Tamaño	Descripción
opcode	1 byte	Valor 3.
numFicheros	4 bytes	Valor 0 en la solicitud.

3.3.2. Respuesta FILE_LIST

Campo	Tamaño	Descripción
opcode	1 byte	Valor 3.
numFicheros	4 bytes	Número de ficheros enviados.
longNombre	2 bytes	Longitud del nombre en UTF-8.
nombre	variable	Nombre del fichero.
tamaño	8 bytes	Tamaño del fichero.
longHash	2 bytes	Longitud del hash en UTF-8.
hash	variable	Hash del fichero.

Los campos desde `longNombre` hasta `hash` se repiten una vez por cada fichero compartido.

3.4. Comando peerdl

El comando `peerdl <peer_nickname><hash_substring>` descarga un fichero desde un *peer* concreto.

El proceso se divide en dos fases:

1. El cliente pregunta al servidor si la subcadena de hash identifica exactamente un fichero. Para ello envía `GET_FILE_HASH`.
2. Si el servidor responde con los metadatos del fichero, el cliente solicita fragmentos con `GET_CHUNK` hasta completar el tamaño total.

3.4.1. Solicitud GET_FILE_HASH

Campo	Tamaño	Descripción
opcode	1 byte	Valor 5.
longHash	2 bytes	Longitud de la subcadena.
hashSubstring	variable	Subcadena del hash.

3.4.2. Respuesta SEND_FILE con metadatos

Campo	Tamaño	Descripción
opcode	1 byte	Valor 4.
longHash	2 bytes	Longitud del hash completo.
hash	variable	Hash completo del fichero.
longNombre	2 bytes	Longitud del nombre.
nombre	variable	Nombre del fichero.
fileSize	8 bytes	Tamaño total del fichero.
fileOffset	8 bytes	Desplazamiento, 0 en la respuesta de metadatos.
dataLength	4 bytes	Valor 0 en la respuesta de metadatos.
data	variable	Vacío en la respuesta de metadatos.

Si la subcadena no coincide con un único fichero, el servidor responde con `FILE_NOT_FOUND`.

3.4.3. Solicitud GET_CHUNK

Campo	Tamaño	Descripción
opcode	1 byte	Valor 2.
longHash	2 bytes	Longitud del hash completo.
hash	variable	Hash completo del fichero solicitado.
fileOffset	8 bytes	Posición desde la que leer.
chunkSize	4 bytes	Número máximo de bytes solicitados.

Este campo **hash** dentro de **GET_CHUNK** es necesario para que el servidor sepa qué fichero debe abrir. Sin este campo, el servidor no podría localizar la ruta del fichero mediante la base de datos local.

3.4.4. Respuesta SEND_FILE con datos

La respuesta con datos usa el mismo formato que la respuesta de metadatos. En este caso, **dataLength** contiene el tamaño real del fragmento enviado y **data** contiene los bytes del fichero.

El cliente escribe cada fragmento con **RandomAccessFile** usando la posición indicada por el campo **fileOffset**. Al terminar, calcula el hash del fichero recibido y lo compara con el hash completo devuelto por el servidor.

3.5. Autómata del peer como cliente TCP

El peer, actuando como cliente TCP, se comunica directamente con otro peer que previamente ha ejecutado **serve**. Este autómata modela las operaciones **peerfiles** y **peerdl**. La obtención de la dirección IP y puerto TCP del peer remoto no forma parte de este autómata, ya que se realiza previamente mediante el protocolo UDP con el directorio.

Al utilizar TCP, el autómata no necesita contemplar retransmisiones ni temporizadores a nivel de aplicación. El transporte ya ofrece entrega fiable y ordenada. Sin embargo, sí se contemplan errores propios del protocolo, como que la conexión no pueda establecerse, que el hash solicitado no exista, que sea ambiguo o que el hash final del fichero descargado no coincida con el esperado.

Los estados del autómata son los siguientes:

- **q0**: estado inicial. El cliente ya conoce la dirección TCP del peer remoto, pero todavía no ha iniciado ninguna operación TCP.
- **qConnect**: estado de establecimiento de conexión. El cliente intenta abrir un socket TCP con el peer servidor.
- **qPeerFiles**: estado de espera de la lista de ficheros tras enviar una petición **FILE_LIST**. Corresponde al comando **peerfiles**.
- **qGetHash**: estado de espera de metadatos del fichero tras enviar **GET_FILE_HASH**. Corresponde al inicio del comando **peerdl**.
- **qChunk_i**: estado de espera del fragmento **i** del fichero tras enviar una petición **GET_CHUNK**. El cliente permanece en este estado mientras queden fragmentos por descargar.
- **qWrite**: estado de escritura en disco del fragmento recibido. Después de escribirlo, el cliente decide si debe solicitar otro fragmento o si ya ha descargado el fichero completo.
- **qVerify**: estado de verificación final. El cliente calcula el hash del fichero descargado y lo compara con el hash completo indicado por el servidor.

- **qFin**: estado final correcto. La operación TCP ha terminado correctamente y la conexión puede cerrarse.
- **qError**: estado final de error. Se alcanza si falla la conexión, si el servidor responde con `FILE_NOT_FOUND`, si se recibe una respuesta inesperada o si el hash final no coincide.

Las transiciones principales del autómata son:

- Desde **q0**, si el usuario ejecuta `peerfiles`, el cliente intenta abrir conexión TCP con el peer remoto mediante la transición hacia **qConnect**.
- Desde **qConnect**, si la conexión TCP se establece correctamente y la operación solicitada es `peerfiles`, el cliente envía `snd(FILE_LIST)` y pasa a **qPeerFiles**.
- Desde **qPeerFiles**, si se recibe `rcv(FILE_LIST)` con la lista de ficheros del servidor, el cliente muestra la información recibida y pasa a **qFin**.
- Desde **q0**, si el usuario ejecuta `peerdl`, el cliente intenta abrir conexión TCP con el peer remoto mediante la transición hacia **qConnect**.
- Desde **qConnect**, si la conexión TCP se establece correctamente y la operación solicitada es `peerdl`, el cliente envía `snd(GET_FILE_HASH)` con la subcadena del hash y pasa a **qGetHash**.
- Desde **qGetHash**, si el servidor responde con `rcv(FILE_NOT_FOUND)`, la operación no puede continuar y se pasa a **qError**. Este caso representa tanto que no exista ningún fichero coincidente como que la subcadena del hash sea ambigua.
- Desde **qGetHash**, si el servidor responde con `rcv(SEND_FILE)` incluyendo el hash completo, el nombre del fichero y su tamaño, el cliente crea el fichero local y pasa a **qChunk_i**.
- Desde **qChunk_i**, el cliente solicita el siguiente fragmento mediante `snd(GET_CHUNK)` indicando el hash completo, el desplazamiento dentro del fichero y el tamaño del fragmento solicitado.
- Desde **qChunk_i**, si se recibe `rcv(SEND_FILE)` con los datos del fragmento solicitado, el cliente pasa a **qWrite**.
- Desde **qWrite**, si todavía quedan bytes pendientes, se actualiza el desplazamiento y se vuelve a **qChunk_i** para solicitar el siguiente fragmento.
- Desde **qWrite**, si ya se ha recibido el fichero completo, el cliente pasa a **qVerify**.
- Desde **qVerify**, si el hash calculado del fichero descargado coincide con el hash esperado, la descarga se considera correcta y se pasa a **qFin**.
- Desde **qVerify**, si el hash calculado no coincide con el hash esperado, se pasa a **qError**.
- Desde cualquier estado de comunicación, si se produce un error de conexión, una respuesta nula o una respuesta con un opcode no esperado, se pasa a **qError**.

3.6. Autómata del peer como servidor TCP

El peer, actuando como servidor TCP, queda a la espera de conexiones entrantes después de ejecutar la orden `serve`. El servidor escucha en un puerto TCP asignado automáticamente y, cada vez que acepta una conexión, crea un hilo de tipo `NFServerThread` para atender a ese cliente. De esta forma, el peer puede atender a varios clientes simultáneamente.

Este autómata modela el comportamiento del servidor de ficheros ante las peticiones `peerfiles` y `peerdl`. Al utilizar TCP, no se incluyen estados de retransmisión ni temporizadores, ya que la fiabilidad del transporte la proporciona TCP.

Los estados del autómata son los siguientes:

- **qListen**: estado inicial del servidor TCP. El peer está escuchando conexiones entrantes en su puerto TCP.
- **qAccept**: estado en el que el servidor acepta una conexión de un cliente y crea un hilo `NFServerThread` para atenderla.
- **qWait**: estado inicial del hilo de atención. El servidor espera recibir un mensaje TCP del cliente.
- **qFileList**: estado de procesamiento de una petición `FILE_LIST`. El servidor obtiene la lista de ficheros compartidos.
- **qGetHash**: estado de procesamiento de una petición `GET_FILE_HASH`. El servidor busca si la subcadena de hash recibida identifica de forma única un fichero compartido.
- **qSendMetadata**: estado en el que el servidor envía un mensaje `SEND_FILE` sin datos de fichero, pero con los metadatos necesarios para iniciar la descarga: nombre, tamaño y hash completo.
- **qGetChunk**: estado de procesamiento de una petición `GET_CHUNK`. El servidor localiza el fichero mediante su hash completo, se posiciona en el desplazamiento indicado y lee el fragmento solicitado.
- **qSendChunk**: estado en el que el servidor envía un mensaje `SEND_FILE` con los bytes del fragmento solicitado.
- **qNotFound**: estado de respuesta negativa. Se alcanza cuando el hash no identifica un único fichero, cuando el fichero no existe o cuando la petición no es válida. El servidor responde con `FILE_NOT_FOUND`.
- **qClose**: estado final de la conexión con ese cliente. El hilo cierra el socket cuando termina la atención o se produce un error.

Las transiciones principales del autómata son:

- Desde **qListen**, cuando llega una nueva conexión TCP, se produce la transición hacia **qAccept**.
- Desde **qAccept**, el servidor crea un hilo `NFServerThread` para el cliente aceptado. El servidor principal vuelve a **qListen**, mientras que el nuevo hilo comienza en **qWait**.
- Desde **qWait**, si se recibe `rcv(FILE_LIST)`, se pasa a **qFileList**. El servidor obtiene los ficheros compartidos y responde con `snd(FILE_LIST)`. Después pasa a **qClose**.
- Desde **qWait**, si se recibe `rcv(GET_FILE_HASH)`, se pasa a **qGetHash**. El servidor busca coincidencias entre la subcadena recibida y los hashes de sus ficheros compartidos.

- Desde **qGetHash**, si la subcadena identifica un único fichero, se pasa a **qSendMetadata**. El servidor responde con **snd(SEND_FILE)** incluyendo el hash completo, el nombre del fichero y su tamaño. Después vuelve a **qWait** para recibir las peticiones de fragmentos.
- Desde **qGetHash**, si no hay coincidencias o hay más de una coincidencia, se pasa a **qNotFound**. El servidor responde con **snd(FILE_NOT_FOUND)** y después pasa a **qClose**.
- Desde **qWait**, si se recibe **rcv(GET_CHUNK)**, se pasa a **qGetChunk**. El servidor comprueba el hash completo, el desplazamiento y el tamaño solicitado.
- Desde **qGetChunk**, si la petición es válida, se pasa a **qSendChunk**. El servidor lee el fragmento correspondiente del fichero y responde con **snd(SEND_FILE)** incluyendo los datos del fragmento.
- Desde **qSendChunk**, el servidor vuelve a **qWait** para permitir que el cliente solicite más fragmentos del mismo fichero.
- Desde **qGetChunk**, si el fichero no existe, el hash no es válido, el desplazamiento está fuera de rango o el tamaño solicitado no es correcto, se pasa a **qNotFound**. El servidor responde con **snd(FILE_NOT_FOUND)** y después pasa a **qClose**.
- Desde cualquier estado del hilo, si se produce un error de entrada/salida o el cliente cierra la conexión, se pasa a **qClose**.

4. Funcionalidad implementada

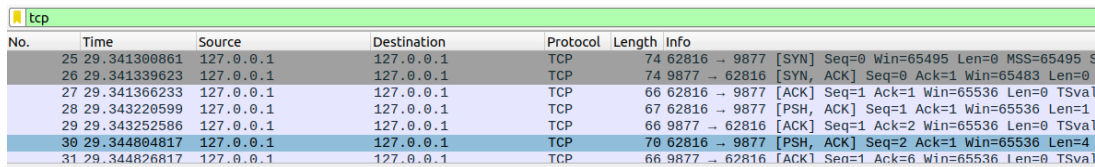
4.1. Funcionalidad obligatoria

Comando	Implementación
ping	Envío UDP con operation:ping y comprobación del protocolo.
dirfiles	Consulta de los ficheros alojados en el directorio, mostrando nombre, tamaño y hash.
peers	Consulta del censo de <i>peers</i> registrados en el directorio.
serve	Arranque del servidor TCP en segundo plano y registro del puerto en el directorio.
peerfiles	Consulta TCP de los ficheros compartidos por un <i>peer</i> concreto.
peerdl <nickname>	Descarga TCP de un fichero desde un <i>peer</i> concreto mediante subcadena de hash.

4.2. Mejoras implementadas

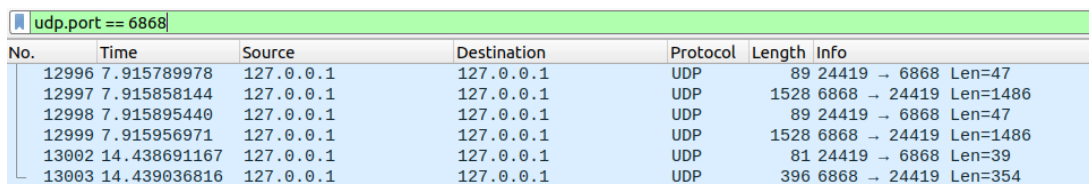
Mejora	Implementación
quit	El peer comunica al directorio que deja de actuar como servidor mediante <code>operation:quit</code> . El directorio elimina su <i>nickname</i> del censo de peers registrados y responde con <code>quitok</code> . Después, el servidor TCP del peer se detiene y la aplicación finaliza.
dirdl básico	El peer solicita al directorio la descarga de un fichero mediante una subcadena de hash. Si la subcadena identifica un único fichero, el directorio responde con <code>downloadok</code> y el cliente guarda el fichero recibido. Si no hay coincidencias o el hash es ambiguo, el directorio responde con <code>error</code> .
dirdl ampliado	La descarga desde el directorio se realiza por bloques numerados mediante el campo <code>numseq</code> . El cliente solicita sucesivamente los fragmentos del fichero y el directorio responde con mensajes <code>downloadok</code> que contienen el bloque correspondiente. Al finalizar, el cliente reconstruye el fichero completo y verifica su integridad comparando el checksum calculado con el hash recibido del directorio.
dirfiles ampliado	El listado de ficheros del directorio puede enviarse dividido en varios datagramas UDP. Cada respuesta <code>dirfilesok</code> incluye la parte correspondiente del listado y permite al cliente reconstruir la salida completa.

5. Capturas de ejecución y Wireshark



No.	Time	Source	Destination	Protocol	Length	Info
25	29.341300861	127.0.0.1	127.0.0.1	TCP	74	62816 → 9877 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 S
26	29.341339623	127.0.0.1	127.0.0.1	TCP	74	9877 → 62816 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0
27	29.341366233	127.0.0.1	127.0.0.1	TCP	66	62816 → 9877 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval
28	29.343220599	127.0.0.1	127.0.0.1	TCP	67	62816 → 9877 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=1
29	29.343252586	127.0.0.1	127.0.0.1	TCP	66	9877 → 62816 [ACK] Seq=1 Ack=2 Win=65536 Len=0 TSval
30	29.344804817	127.0.0.1	127.0.0.1	TCP	70	62816 → 9877 [PSH, ACK] Seq=2 Ack=1 Win=65536 Len=4
31	29.344826817	127.0.0.1	127.0.0.1	TCP	66	9877 → 62816 [ACK] Seq=1 Ack=6 Win=65536 Len=0 TSval

Figura 1: Captura en Wireshark del intercambio TCP del comando `peerfiles`. Se observa el establecimiento de conexión entre peers mediante el three-way handshake y el posterior intercambio de datos.



No.	Time	Source	Destination	Protocol	Length	Info
12996	7.915789978	127.0.0.1	127.0.0.1	UDP	89	24419 → 6868 Len=47
12997	7.915858144	127.0.0.1	127.0.0.1	UDP	1528	6868 → 24419 Len=1486
12998	7.915895440	127.0.0.1	127.0.0.1	UDP	89	24419 → 6868 Len=47
12999	7.915956971	127.0.0.1	127.0.0.1	UDP	1528	6868 → 24419 Len=1486
13002	14.438691167	127.0.0.1	127.0.0.1	UDP	81	24419 → 6868 Len=39
13003	14.439036816	127.0.0.1	127.0.0.1	UDP	396	6868 → 24419 Len=354

Figura 2: Captura en Wireshark del intercambio UDP del comando `dirfiles`. El peer solicita al directorio la lista de ficheros disponibles y el servidor responde a través del puerto 6868.

udp.port == 6868						
No.	Time	Source	Destination	Protocol	Length	Info
468	32.328909262	127.0.0.1	127.0.0.1	UDP	88	24419 → 6868 Len=46
469	32.328988603	127.0.0.1	127.0.0.1	UDP	1544	6868 → 24419 Len=1502
470	32.329099722	127.0.0.1	127.0.0.1	UDP	88	24419 → 6868 Len=46
471	32.329274817	127.0.0.1	127.0.0.1	UDP	1544	6868 → 24419 Len=1502
472	32.329395924	127.0.0.1	127.0.0.1	UDP	88	24419 → 6868 Len=46
473	32.329527856	127.0.0.1	127.0.0.1	UDP	1544	6868 → 24419 Len=1502

Figura 3: Captura en Wireshark del intercambio UDP con el directorio durante el comando `dirdl`. Se observa la comunicación entre el peer y el servidor de directorio en el puerto 6868, con respuestas fragmentadas del directorio.

6. Pruebas realizadas

Durante las pruebas se ha utilizado el siguiente flujo de ejecución:

1. Lanzar `Directory` para que el directorio escuche en el puerto UDP 6868.
2. Lanzar uno o varios procesos `NanoFiles` indicando la IP o nombre del host donde se ejecuta el directorio.
3. Ejecutar `ping` para pasar el *peer* a estado `ONLINE`.
4. Ejecutar `serve` para registrar un *peer* como servidor de ficheros.
5. Ejecutar `dirfiles` para obtener la información de los ficheros que contiene el Directorio.
6. Ejecutar `dirdl <subHash>` para descargar un fichero perteneciente al Directorio a partir de su subHash.
7. Ejecutar `peers` para comprobar que el directorio contiene el *nickname*, la IP y el puerto TCP del servidor.
8. Ejecutar `peerfiles <nickname>` para listar los ficheros compartidos por el servidor.
9. Ejecutar `peerdl <nickname><hash_substring>` para descargar un fichero desde el servidor TCP.
10. Comprobar que el fichero se guarda con el nombre remoto y que el hash calculado coincide con el hash esperado.

Enlace al video de prueba del proyecto NanoFiles: <https://youtu.be/NG65Lkd-1rs>

7. Conclusiones

Tras realizar la implementación de este proyecto podemos afirmar que hemos comprendido mucho mejor tanto el uso como las características de dos protocolos de transporte fundamentales como lo son UDP y TCP. Durante el desarrollo del proyecto nos hemos encontrado con distintos problemas, los cuales nos han permitido profundizar en la realidad de las comunicaciones en red.

En primer lugar, la comunicación con el Directorio nos ha servido para entender las limitaciones de un protocolo no confiable como UDP. El reto de implementar `dirfiles` ampliado y `dirdl` ampliado nos ha obligado a diseñar mecanismos de fragmentación, reconstrucción y control de flujo mediante el protocolo de Parada y Espera.

Por otro lado, la implementación del protocolo entre pares (P2P) sobre TCP nos ha mostrado las ventajas de un canal orientado a la conexión y confiable por naturaleza. En esta fase, nos hemos centrado en el diseño de mensajes binarios multiformato, donde el uso de *opcodes* ha sido

esencial para optimizar el ancho de banda y permitir la transferencia de estructuras de datos complejas como listas de ficheros y fragmentos binarios.

Por último, este proyecto no solo nos ha permitido comprobar las cuestiones teóricas estudiadas en la asignatura, sino que también hemos obtenido los conocimientos prácticos para poder comprender de una forma mucho más específica las Redes de Comunicaciones.